BRIGHTLOOP SOFTWARE

Database & Backend Implementation Guide

PostgreSQL · Node.js · Express · pg (node-postgres)

Purpose
An implementation-oriented database reference for building Brightloop's subscription management platform. It covers the relational model, core SQL structures, billing integrity, idempotency, transactions, indexes, and safe database access from Express.

Architecture decision: React → Express API → PostgreSQL via pg connection pool. Direct SQL; no ORM.

Prepared for backend implementation · Version 1.0 · 31 October 2026

1. How to use this guide

Build the backend in layers. Start with the database and connection pool, then implement authentication and tenant isolation, catalog and plans, quotations, subscriptions, billing, payments, and reporting. Do not implement every table and endpoint in one pass.

The schema below is a proposed implementation design based on the Brightloop requirements discussed so far. Some business policies were not specified in the requirements; those are marked as decisions to confirm rather than silently assumed.

Non-negotiable engineering rules

• **Money:** use NUMERIC/DECIMAL, never JavaScript floating-point values as the authoritative stored amount.

• **Time:** use TIMESTAMPTZ for events and DATE for date-only business fields.

• **Passwords:** store password hashes only (Argon2id or bcrypt), never plaintext.

• **Financial history:** do not hard-delete invoices, payments, subscriptions, or audit records.

• **Invoice snapshots:** copy product description, unit price, tax rate, discount, and totals onto invoice lines.

• **Concurrency:** use database constraints and transactions; application checks alone are not sufficient.

• **Tenant isolation:** derive customer_id from the authenticated user, never trust a customer_id sent by the browser.

Important: the DDL in this guide is a practical starting point, not a substitute for confirming currency, tax, cancellation, proration, and payment-provider policies.

2. System architecture

Layer	Responsibility
React client	Calls versioned REST endpoints; never connects directly to PostgreSQL.
Express API entry point	Routes, authentication, authorization, validation, rate limits, request IDs, and consistent responses. This is the initial API gateway/entry point.
Controllers	Translate HTTP requests into service calls; avoid business rules and raw SQL here.
Services	Business workflows and transaction boundaries: confirm quotation, create subscription, generate invoice, apply payment.
Repositories	Parameterized SQL through pg; return rows/results; no HTTP response handling.
PostgreSQL	Source of truth: constraints, foreign keys, unique indexes, transactions, and durable financial records.

Do not introduce microservices or a separate gateway product at the beginning. Keep a modular monolith in Express. A dedicated gateway (for example Kong or NGINX) can be added later if deployment topology, traffic, or multiple services justify it.

3. Entity inventory

Area	Tables	Purpose
Identity	customers, users, password_reset_tokens	Customer companies, staff/admin accounts, customer logins, reset flows.
Catalog	products, product_variants	Sellable products and optional variants.
Plans	plans, plan_items	Recurring plan configuration and included products.
Quotations	quotation_templates, quotation_template_items, quotations, quotation_items	Reusable templates and customer-specific commercial offers.

Area	Tables	Purpose
Subscriptions	subscriptions, subscription_items	Contract lifecycle, billing schedule, and subscribed lines.
Billing	invoices, invoice_items, billing_runs	Invoices, immutable line snapshots, and safe recurring billing runs.
Payments	payments, payment_refunds	Payment attempts/receipts and refund history.
Discounts	discounts, discount_products, discount_usages	Admin-managed discount rules and usage tracking.
Tax & controls	taxes, audit_logs, idempotency_keys	Tax configuration, audit history, and duplicate-request protection.

4. Relationships at a glance

• One customer company can have many individual users, quotations, subscriptions, and invoices.

• One product can have many variants. A plan has many plan_items; each item references a product and optionally a variant.

• A quotation has many quotation_items. An accepted quotation may create at most one subscription.

• A subscription has many subscription_items and invoices. An invoice has many invoice_items and payments.

• A discount can apply to many products; discount_products is the many-to-many junction table.

• A payment may have multiple refund records. Refunds do not overwrite the original payment.

Company users are separate login identities (unique user IDs and emails) that share the same customer_id. Never use shared company credentials. A customer user's queries must be scoped to the customer_id stored on their authenticated user record.

5. Core PostgreSQL schema

The following DDL illustrates the main structures and constraints. Use UUID primary keys generated by the application or gen_random_uuid() where available. Apply migrations in small, ordered files and test each migration against a clean database.

5.1 Identity and customer accounts

```
CREATE TABLE customers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_name TEXT NOT NULL,
  billing_email TEXT,
  phone TEXT,
  billing_address JSONB,
  status TEXT NOT NULL DEFAULT 'active'
    CHECK (status IN ('active','inactive','archived')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id UUID REFERENCES customers(id),
  full_name TEXT NOT NULL,
  email TEXT NOT NULL,
  password_hash TEXT NOT NULL,
  role TEXT NOT NULL CHECK (role IN ('admin','staff','customer')),
  is_active BOOLEAN NOT NULL DEFAULT true,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT users_email_unique UNIQUE (email),
  CONSTRAINT customer_role_scope CHECK (
    (role = 'customer' AND customer_id IS NOT NULL)
    OR (role IN ('admin','staff') AND customer_id IS NULL)
  )
);

CREATE TABLE password_reset_tokens (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id),
```

```
  token_hash TEXT NOT NULL UNIQUE,
  expires_at TIMESTAMPTZ NOT NULL,
  used_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Normalize email addresses (for example, lowercase and trim) before insertion. Consider a case-insensitive unique index if email normalization cannot be guaranteed by the application.

5.2 Products, variants, plans

```
CREATE TABLE products (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  product_type TEXT NOT NULL,
  sales_price NUMERIC(12,2) NOT NULL CHECK (sales_price >= 0),
  cost_price NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (cost_price >= 0),
  notes TEXT,
  is_active BOOLEAN NOT NULL DEFAULT true,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE product_variants (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  product_id UUID NOT NULL REFERENCES products(id),
  name TEXT NOT NULL,
  additional_price NUMERIC(12,2) NOT NULL DEFAULT 0,
  is_active BOOLEAN NOT NULL DEFAULT true,
  UNIQUE (product_id, name)
);

CREATE TABLE plans (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  description TEXT,
  base_price NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (base_price >= 0),
  billing_frequency TEXT NOT NULL
    CHECK (billing_frequency IN ('daily','weekly','monthly','yearly')),
  minimum_quantity INTEGER NOT NULL DEFAULT 1 CHECK (minimum_quantity > 0),
  available_from DATE,
  available_until DATE,
  auto_close BOOLEAN NOT NULL DEFAULT false,
  allow_early_closure BOOLEAN NOT NULL DEFAULT true,
  allow_pause BOOLEAN NOT NULL DEFAULT false,
  allow_renewal BOOLEAN NOT NULL DEFAULT true,
  is_active BOOLEAN NOT NULL DEFAULT true,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CHECK (available_until IS NULL OR available_from IS NULL
         OR available_until >= available_from)
);

CREATE TABLE plan_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  plan_id UUID NOT NULL REFERENCES plans(id),
  product_id UUID NOT NULL REFERENCES products(id),
  variant_id UUID REFERENCES product_variants(id),
  quantity INTEGER NOT NULL DEFAULT 1 CHECK (quantity > 0),
  unit_price NUMERIC(12,2) NOT NULL CHECK (unit_price >= 0)
);
```

The plan price model needs a business decision: is base_price the complete bundle price, or is it added to the sum of plan_items? Keep that rule consistent in quotation, subscription, and invoice calculations.

5.3 Quotations

```
CREATE TABLE quotation_templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  validity_days INTEGER NOT NULL CHECK (validity_days > 0),
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE quotation_template_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  template_id UUID NOT NULL REFERENCES quotation_templates(id),
  product_id UUID REFERENCES products(id),
  plan_id UUID REFERENCES plans(id),
  quantity INTEGER NOT NULL DEFAULT 1 CHECK (quantity > 0),
  unit_price NUMERIC(12,2) NOT NULL CHECK (unit_price >= 0),
```

```
  CHECK (product_id IS NOT NULL OR plan_id IS NOT NULL)
);

CREATE TABLE quotations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  quotation_number TEXT NOT NULL UNIQUE,
  customer_id UUID NOT NULL REFERENCES customers(id),
  status TEXT NOT NULL DEFAULT 'draft'
    CHECK (status IN ('draft','sent','accepted','rejected','expired','cancelled')),
  issue_date DATE NOT NULL DEFAULT CURRENT_DATE,
  valid_until DATE NOT NULL,
  subtotal NUMERIC(12,2) NOT NULL DEFAULT 0,
  tax_total NUMERIC(12,2) NOT NULL DEFAULT 0,
  discount_total NUMERIC(12,2) NOT NULL DEFAULT 0,
  total NUMERIC(12,2) NOT NULL DEFAULT 0,
  created_by UUID REFERENCES users(id),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE quotation_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  quotation_id UUID NOT NULL REFERENCES quotations(id),
  product_id UUID REFERENCES products(id),
  plan_id UUID REFERENCES plans(id),
  description TEXT NOT NULL,
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  unit_price NUMERIC(12,2) NOT NULL CHECK (unit_price >= 0),
  tax_rate NUMERIC(7,4) NOT NULL DEFAULT 0,
  line_total NUMERIC(12,2) NOT NULL,
  CHECK (product_id IS NOT NULL OR plan_id IS NOT NULL)
);
```

5.4 Subscriptions

```
CREATE TABLE subscriptions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  subscription_number TEXT NOT NULL UNIQUE,
  customer_id UUID NOT NULL REFERENCES customers(id),
  plan_id UUID REFERENCES plans(id),
  source_quotation_id UUID UNIQUE REFERENCES quotations(id),
  status TEXT NOT NULL DEFAULT 'draft'
    CHECK (status IN ('draft','quotation','confirmed','active','paused','closed')),
  start_date DATE,
  expiry_date DATE,
  next_billing_at TIMESTAMPTZ,
  payment_terms_days INTEGER NOT NULL DEFAULT 0
    CHECK (payment_terms_days >= 0),
  currency CHAR(3) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE subscription_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  subscription_id UUID NOT NULL REFERENCES subscriptions(id),
  product_id UUID REFERENCES products(id),
  variant_id UUID REFERENCES product_variants(id),
  description TEXT NOT NULL,
  quantity INTEGER NOT NULL CHECK (quantity > 0),
  unit_price NUMERIC(12,2) NOT NULL CHECK (unit_price >= 0),
  tax_rate NUMERIC(7,4) NOT NULL DEFAULT 0
);
```

source_quotation_id UNIQUE prevents one accepted quotation from creating multiple subscriptions. Keep it nullable for subscriptions created without a quotation.

5.5 Taxes, invoices, and invoice line snapshots

```
CREATE TABLE taxes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  rate NUMERIC(7,4) NOT NULL CHECK (rate >= 0),
  is_active BOOLEAN NOT NULL DEFAULT true,
  effective_from DATE,
  effective_until DATE
);

CREATE TABLE invoices (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  invoice_number TEXT NOT NULL UNIQUE,
  customer_id UUID NOT NULL REFERENCES customers(id),
  subscription_id UUID NOT NULL REFERENCES subscriptions(id),
  billing_period_start DATE NOT NULL,
```

```
  billing_period_end DATE NOT NULL,
  issue_date DATE NOT NULL DEFAULT CURRENT_DATE,
  due_date DATE NOT NULL,
  status TEXT NOT NULL DEFAULT 'draft'
    CHECK (status IN ('draft','confirmed','paid','cancelled')),
  currency CHAR(3) NOT NULL,
  subtotal NUMERIC(12,2) NOT NULL DEFAULT 0,
  discount_total NUMERIC(12,2) NOT NULL DEFAULT 0,
  tax_total NUMERIC(12,2) NOT NULL DEFAULT 0,
  total NUMERIC(12,2) NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (subscription_id, billing_period_start, billing_period_end),
  CHECK (billing_period_end >= billing_period_start)
);

CREATE TABLE invoice_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  invoice_id UUID NOT NULL REFERENCES invoices(id),
  product_id UUID REFERENCES products(id),
  description_snapshot TEXT NOT NULL,
  quantity NUMERIC(12,3) NOT NULL CHECK (quantity > 0),
  unit_price NUMERIC(12,2) NOT NULL,
  discount_amount NUMERIC(12,2) NOT NULL DEFAULT 0,
  tax_rate_snapshot NUMERIC(7,4) NOT NULL DEFAULT 0,
  tax_amount NUMERIC(12,2) NOT NULL DEFAULT 0,
  line_total NUMERIC(12,2) NOT NULL
);
```

Invoice lines deliberately snapshot descriptions and prices. If a product is renamed or its price/tax changes next month, an old invoice must still show the values agreed at the time it was issued.

5.6 Payments and refunds

```
CREATE TABLE payments (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  invoice_id UUID NOT NULL REFERENCES invoices(id),
  provider TEXT,
  provider_payment_id TEXT,
  idempotency_key TEXT,
  method TEXT NOT NULL,
  amount NUMERIC(12,2) NOT NULL CHECK (amount > 0),
  currency CHAR(3) NOT NULL,
  status TEXT NOT NULL
    CHECK (status IN ('pending','succeeded','failed','cancelled')),
  paid_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (provider, provider_payment_id)
);

CREATE UNIQUE INDEX payments_idempotency_key_unique
ON payments(idempotency_key)
WHERE idempotency_key IS NOT NULL;

CREATE TABLE payment_refunds (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  payment_id UUID NOT NULL REFERENCES payments(id),
  provider_refund_id TEXT UNIQUE,
  amount NUMERIC(12,2) NOT NULL CHECK (amount > 0),
  status TEXT NOT NULL
    CHECK (status IN ('pending','succeeded','failed')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

The unique provider payment ID is a webhook/reconciliation safeguard. A refund is a separate financial event; do not edit the original payment amount to represent a refund.

5.7 Discounts and usage

```
CREATE TABLE discounts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  discount_type TEXT NOT NULL CHECK (discount_type IN ('fixed','percentage')),
  value NUMERIC(12,2) NOT NULL CHECK (value > 0),
  minimum_purchase NUMERIC(12,2) NOT NULL DEFAULT 0,
  minimum_quantity INTEGER NOT NULL DEFAULT 0,
  starts_at TIMESTAMPTZ,
  ends_at TIMESTAMPTZ,
  usage_limit INTEGER,
  usage_count INTEGER NOT NULL DEFAULT 0,
  applies_to TEXT NOT NULL CHECK (applies_to IN ('products','subscription')),
  is_active BOOLEAN NOT NULL DEFAULT true,
  CHECK (usage_limit IS NULL OR usage_limit >= 0),
```

```
  CHECK (ends_at IS NULL OR starts_at IS NULL OR ends_at > starts_at)
);

CREATE TABLE discount_products (
  discount_id UUID NOT NULL REFERENCES discounts(id),
  product_id UUID NOT NULL REFERENCES products(id),
  PRIMARY KEY (discount_id, product_id)
);

CREATE TABLE discount_usages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  discount_id UUID NOT NULL REFERENCES discounts(id),
  customer_id UUID NOT NULL REFERENCES customers(id),
  invoice_id UUID REFERENCES invoices(id),
  subscription_id UUID REFERENCES subscriptions(id),
  amount_applied NUMERIC(12,2) NOT NULL CHECK (amount_applied >= 0),
  applied_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

To enforce a usage cap safely, lock the discount row inside the same transaction that creates the invoice and usage record, or use a conditional atomic UPDATE. Do not read usage_count, check it in JavaScript, and increment later without locking.

5.8 Idempotency, billing runs, and audit logs

```
CREATE TABLE idempotency_keys (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  key TEXT NOT NULL,
  scope TEXT NOT NULL,
  request_hash TEXT NOT NULL,
  response_status INTEGER,
  response_body JSONB,
  resource_id UUID,
  status TEXT NOT NULL DEFAULT 'processing'
    CHECK (status IN ('processing','completed','failed')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at TIMESTAMPTZ,
  UNIQUE (scope, key)
);

CREATE TABLE billing_runs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  started_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  finished_at TIMESTAMPTZ,
  status TEXT NOT NULL CHECK (status IN ('running','completed','failed')),
  processed_count INTEGER NOT NULL DEFAULT 0,
  error_summary TEXT
);

CREATE TABLE audit_logs (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  actor_user_id UUID REFERENCES users(id),
  action TEXT NOT NULL,
  entity_type TEXT NOT NULL,
  entity_id UUID,
  metadata JSONB,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

6. Idempotency: how to prevent duplicate work

Idempotency means that repeating the same logical request does not repeat its side effect. It is essential for payments, invoice generation, quotation confirmation, subscription creation, and payment webhooks.

Request flow

- Client sends an Idempotency-Key header for a supported write operation.
- Server validates the request and computes a stable request hash.
- Insert (scope, key) into idempotency_keys. The unique constraint arbitrates concurrent requests.
- If the same key already completed with the same request hash, return the stored status/body.
- If the same key is reused with a different request hash, reject it (typically HTTP 409).
- Perform the business operation and save the response/resource reference in the same transaction.

• If the transaction fails, roll back. Define a safe retry policy for failed/in-progress records.

Use a scope such as `POST:/api/v1/invoices` plus the authenticated actor/tenant scope where appropriate. Do not use one global key namespace across unrelated operations.

Database-level safeguards that must also exist

Operation	Durable protection
Invoice per billing period	UNIQUE(subscription_id, billing_period_start, billing_period_end)
Quotation → subscription	UNIQUE(source_quotation_id)
Provider webhook	UNIQUE(provider, provider_event_id) in a dedicated webhook_events table, or equivalent provider event ledger.
Payment request	Unique idempotency key and provider payment ID.
Discount usage cap	Row lock / conditional update inside transaction.

Idempotency keys are not a replacement for business uniqueness constraints. Both are required: one protects retries; the other protects the underlying business invariant.

7. Transactions for financial workflows

7.1 Generate a recurring invoice

```
const client = await pool.connect();

try {
  await client.query("BEGIN");

  // Lock the subscription so two workers cannot bill it simultaneously.
  const subResult = await client.query(
    `SELECT * FROM subscriptions
     WHERE id = $1
     FOR UPDATE`,
    [subscriptionId]
  );

  if (subResult.rowCount === 0) {
    throw new Error("Subscription not found");
  }

  // Calculate the period and totals using server-side business rules.
  // Insert invoice; unique billing-period constraint is the final guard.
  // Insert invoice_items snapshots.
  // Advance next_billing_at in this same transaction.

  await client.query("COMMIT");
} catch (error) {
  await client.query("ROLLBACK");
  throw error;
} finally {
  client.release();
}
```

In production, handle a unique-constraint conflict for the billing period as an already-generated invoice, then return the existing invoice where appropriate. Never advance next_billing_at in a separate transaction from invoice creation.

7.2 Payment recording

• Verify the payment provider's webhook signature before trusting the event.

• Insert the provider event ID into a unique event ledger so the same event cannot be processed twice.

• Lock the invoice row before recalculating its outstanding balance.

• Insert the payment and update invoice status only when the payment is confirmed according to provider evidence.

• Commit the event ledger, payment, and invoice state together.

Do not mark an invoice paid merely because the browser says payment succeeded. Trust a verified server-to-server provider event or a server-side provider verification response.

8. Async/await and try/catch database access

Every asynchronous database call must be awaited. Controllers should pass unexpected errors to centralized Express error middleware. Repository functions should use parameterized SQL and should not construct SQL by concatenating user input.

8.1 Pool configuration

```
import pg from "pg";
import dotenv from "dotenv";

dotenv.config();
const { Pool } = pg;

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  max: 10,
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 5000
});

pool.on("error", (err) => {
  console.error("Unexpected idle PostgreSQL client error", err);
});

export default pool;
```

8.2 Repository function

```
import pool from "../config/db.js";

export async function findCustomerById(customerId) {
  try {
    const result = await pool.query(
      `SELECT id, company_name, billing_email, status
       FROM customers
       WHERE id = $1`,
      [customerId]
    );

    return result.rows[0] ?? null;
  } catch (error) {
    // Log context safely; do not expose SQL or secrets to clients.
    console.error("findCustomerById failed:", error);
    throw error;
  }
}
```

You do not need a try/catch in every tiny function if it adds no value. A clean pattern is: repository awaits SQL and throws; service manages business logic/transactions; controller catches and calls next(error); centralized middleware formats the HTTP response. Never silently swallow an error.

8.3 Controller and error middleware

```
export async function getCustomer(req, res, next) {
  try {
    const customer = await findCustomerById(req.params.id);

    if (!customer) {
      return res.status(404).json({
        success: false,
        message: "Customer not found"
      });
    }

    return res.status(200).json({
      success: true,
      data: customer
    });
  } catch (error) {
    return next(error);
  }
}
```

```
// Register after all routes:
app.use((err, req, res, next) => {
  console.error(err);

  const status = err.statusCode || 500;
  return res.status(status).json({
    success: false,
    message: status === 500 ? "Internal server error" : err.message
  });
});
```

8.4 Transaction helper

```
export async function withTransaction(callback) {
  const client = await pool.connect();

  try {
    await client.query("BEGIN");
    const result = await callback(client);
    await client.query("COMMIT");
    return result;
  } catch (error) {
    try {
      await client.query("ROLLBACK");
    } catch (rollbackError) {
      console.error("Rollback failed:", rollbackError);
    }
    throw error;
  } finally {
    client.release();
  }
}
```

Every query inside a transaction must use the same `client`, not `pool.query()`. Always release the client in `finally`. Do not return success before COMMIT succeeds.

9. API naming and gateway conventions

Action	HTTP method + route
List products	GET /api/v1/products
Create product	POST /api/v1/products
Get one product	GET /api/v1/products/:productId
Update product	PATCH /api/v1/products/:productId
Archive product	DELETE /api/v1/products/:productId (soft-delete/archive semantics)
Confirm quotation	POST /api/v1/quotations/:quotationId/confirm
Pause subscription	POST /api/v1/subscriptions/:subscriptionId/pause
Generate invoice	POST /api/v1/subscriptions/:subscriptionId/invoices (if manually triggered)
Record payment	POST /api/v1/invoices/:invoiceId/payments

Use plural nouns for collections, camelCase for JavaScript variables, snake_case for PostgreSQL tables/columns, and stable resource IDs in URLs. Use `/api/v1` from the first endpoint. Avoid verbs in ordinary CRUD routes; explicit action routes are appropriate for state transitions.

Consistent response shape

```
{
  "success": true,
  "data": { },
  "meta": { "page": 1, "limit": 20, "total": 125 }
}

// Error:
{
  "success": false,
  "message": "Validation failed",
  "code": "VALIDATION_ERROR",
```

```
  "details": []
}
```

Express is the initial API gateway/entry point. Apply authentication, authorization, request validation, rate limiting, request IDs, CORS policy, security headers, and routing there. Do not trust the gateway alone: services and PostgreSQL constraints still enforce business rules.

10. Indexes and performance

The target is under two seconds per page response with thousands of subscriptions. This is a goal to measure, not a guarantee provided by indexes alone.

```
CREATE INDEX users_email_idx ON users (email);
CREATE INDEX subscriptions_customer_status_idx
  ON subscriptions (customer_id, status);
CREATE INDEX subscriptions_due_idx
  ON subscriptions (next_billing_at)
  WHERE status = 'active';
CREATE INDEX invoices_customer_due_idx
  ON invoices (customer_id, due_date);
CREATE INDEX invoices_subscription_idx ON invoices (subscription_id);
CREATE INDEX payments_invoice_created_idx
  ON payments (invoice_id, created_at DESC);
CREATE INDEX discount_usages_discount_idx
  ON discount_usages (discount_id, applied_at DESC);
CREATE INDEX audit_logs_entity_idx
  ON audit_logs (entity_type, entity_id, created_at DESC);
```

• Paginate list endpoints; do not return every invoice/subscription in one response.

• Select only the columns required by the page.

• Use indexes that match actual WHERE, JOIN, and ORDER BY patterns.

• Use EXPLAIN (ANALYZE, BUFFERS) with representative data to verify slow queries.

• Aggregate dashboard metrics in SQL; avoid fetching thousands of rows just to total them in Node.

• Consider keyset pagination for large or frequently changing lists.

11. Security and data integrity checklist

• Use parameterized SQL (`$1`, `$2`, ...); never interpolate user values into SQL strings.

• Customer portal queries must use authenticated `req.user.customer_id` as the tenant boundary.

• Enforce admin-only staff creation and discount creation in authorization middleware and service logic.

• Hash passwords with Argon2id or bcrypt; rate-limit login and password-reset endpoints.

• Store only a hash of password-reset tokens; expire and consume tokens once.

• Validate all request bodies and path parameters before business logic.

• Do not expose database error messages, stack traces, secrets, or SQL text to clients.

• Use TLS for production database connections and keep credentials in environment/secret management.

• Use soft deletion/archive for records referenced by financial history.

• Write audit events for sensitive changes: role changes, discount creation, invoice cancellation, refunds, and subscription closure.

12. Backend implementation order

Milestone	Implement	Acceptance check
1. Foundation	PostgreSQL database, migrations, pg pool, Express app/server, health route.	Server starts; database health check succeeds; connection errors are handled.

Milestone	Implement	Acceptance check
2. Identity	Users, customers, password hashing, login, reset tokens, roles.	Unique email; reset token expires/one-use; protected routes work.
3. Tenant isolation	Customer scope middleware and repository query rules.	A customer cannot read another company's records by changing an ID.
4. Catalog	Products, variants, plans, plan items.	CRUD works with validation, pagination, and constraints.
5. Quotations	Templates, quotation lines, status transitions.	Quotation totals are consistent; confirmation is safe to retry.
6. Subscriptions	Subscription creation, lifecycle, items, pause/close rules.	One accepted quotation creates at most one subscription.
7. Billing	Billing worker/run, invoice snapshots, due dates, idempotency.	Retrying a billing run cannot create duplicate invoices.
8. Payments	Provider integration, webhook verification, payment/refund ledger.	Duplicate webhooks do not duplicate payments.
9. Discounts	Eligibility, product scope, usage caps.	Concurrent requests cannot exceed the configured usage cap.
10. Dashboard	Revenue, outstanding, overdue, active subscription aggregates.	Totals reconcile against invoice/payment records.

13. Business decisions to confirm before production

Decision	Why it matters
Currency	Invoice, payment, and rounding rules; decide whether one company can use multiple currencies.
Billing timezone	Defines the meaning of a billing date and when a recurring invoice is due.
Tax rules	Tax inclusive/exclusive, jurisdiction, effective dates, and whether tax applies per line or invoice.
Plan pricing	Whether plan base price is a bundle total or added to plan item prices.
Cancellation/proration	Whether early closure refunds unused time or charges through period end.
Pause/resume	Whether pause shifts the next billing date or preserves the original schedule.
Failed renewals	Retry schedule, grace period, and when to suspend/close a subscription.
Discount stacking	Whether multiple discounts can apply to the same invoice.
Payment allocation	Whether overpayments become customer credit and how partial payments work.
Staff permissions	Exact actions staff may perform versus admin-only actions.

14. Final pre-implementation checklist

- Create a migration folder and keep every schema change in a numbered SQL migration.
- Run migrations against a fresh local PostgreSQL database.
- Seed only development/test data; never hard-code production secrets.
- Add tests for unique constraints, foreign keys, tenant isolation, idempotency, and rollback behavior.
- Test two concurrent invoice-generation requests for the same subscription period.
- Test duplicate payment webhooks and duplicate client requests.
- Test partial payment, refund, cancellation, and discount-cap scenarios.
- Check query plans and response times with realistic data before claiming the performance target is met.

Implementation principle: Express organizes and validates the request; services coordinate business workflows; repositories execute parameterized SQL; PostgreSQL constraints and transactions protect the data. This separation keeps the backend understandable as Brightloop grows.